

Abstract

This project presents the early design and development of a low-cost smart home prototype built using an Arduino Uno R3 and a Keystudio Smart Home Kit. The goal is to explore how a network of security and environmental sensors can automate household functions, promote safety, and demonstrate embedded-system engineering principles. The system integrates a variety of sensors, multiple actuators, and a simple user-interface to simulate a real world functioning smart home environment. The prototype establishes a clear foundation for automation-logic, real time sensing, and a future expansion toward Internet-of-Things (IOT) capabilities.

Introduction

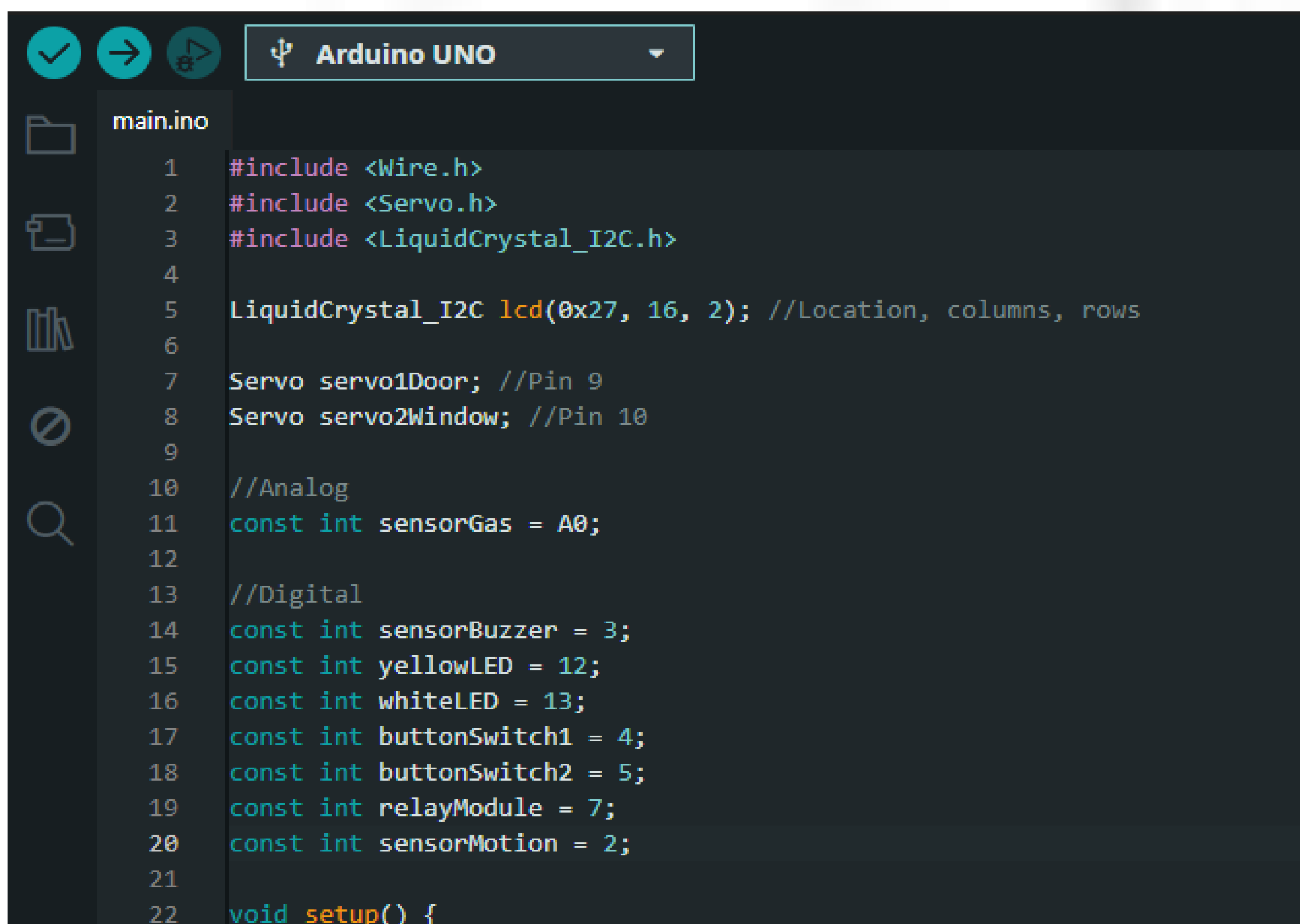
Smart home systems combine sensors, actuators, and microcontrollers to automate everyday household tasks. These systems typically monitor environmental conditions, detect hazards, and respond automatically to user-defined conditions. This project uses the Keystudio Smart Home Kit to prototype a small-scale system capable of sensing motion, detecting gas/hazardous odors, controlling lights, and displaying real-time information. The design emphasizes accessibility, low cost, and modularity, making it useable for beginners while still demonstrating core embedded-system concepts.

Background

A smart home system relies on three core components. First is sensors, which are used to collect information about the environment. Next are actuators, which are used to perform actions based on sensor data. Last is control logic, which is used to interpret data and trigger system responses.

The Arduino Uno R3 provides a simple and flexible platform for integrating these components. The Keystudio smart home kit includes prefabricated modules such as motion sensors, gas sensors, LEDs, passive piezo buzzers, and relay’s that simplify the wiring process and accelerate development. This project focuses on building a functional, modular system that demonstrates how these components interact to create automated home behavior.

Figure 1. Sample code for storing modules to assigned GPIO pins:



```
1  #include <Wire.h>
2  #include <Servo.h>
3  #include <LiquidCrystal_I2C.h>
4
5  LiquidCrystal_I2C lcd(0x27, 16, 2); //Location, columns, rows
6
7  Servo servo1Door; //Pin 9
8  Servo servo2Window; //Pin 10
9
10 //Analog
11 const int sensorGas = A0;
12
13 //Digital
14 const int sensorBuzzer = 3;
15 const int yellowLED = 12;
16 const int whiteLED = 13;
17 const int buttonSwitch1 = 4;
18 const int buttonSwitch2 = 5;
19 const int relayModule = 7;
20 const int sensorMotion = 2;
21
22 void setup() {
```

Method

The system was developed using Arduino IDE and programmed in C++ based Arduino code. Each module was wired to the Arduino Uno individually and tested before being integrated into the full system.

Hardware Components

- **PIR Motion sensor:** Used to detect movement within a proximity and triggers security alerts.
- **Gas / Hazardous odor sensor:** Used to monitor air quality and identify hazardous fumes.
- **Relay Module:** Controls external devices such as a fan and has ability to maintain different voltages. Can also be used to control lights.
- **LED indicators:** Used to provide visual status updates.
- **Passive Piezo Buzzer:** Used to emit audible warnings. This buzzer specifically can output the sound of any musical note, therefore can be used to play familiar sound patterns.
- **Buttons/Switches:** Used to allow manual mode changes.
- **I2C LCD Display:** Used to show real-time sensor readings and the state of system components.

System Logic

- Continuously reading sensor values
- Comparing readings to pre-defined specifications
- Triggering automated responses (LEDs, alarms, messages)
- Displaying the system status on LCD display
- Allowing users to override and make decisions with buttons

This approach ensures the system can behave predictably and will respond immediately to environmental changes.

Figure 2. Shows components are wired correctly and receiving a connection while waiting to be programmed.



Results

In result, the system operates as a fully functional early-stage prototype. It meets the primary goals of real-time environmental monitoring, automated safety responses, basic user interaction and a clear readable system output. Overall, the project shows that a low-cost microcontroller platform can replicate essential smart home behaviors without requiring advanced and expensive hardware.

Figure 3. Shows a successful connection with the I2C LCD, allowing you to see an example of how sensor readings are displayed.



Findings

The prototype successfully demonstrates the core functions expected of a basic smart home system. This includes a motion detection that reliably triggers alerts, gas sensor readings that respond to environmental changes, a relay module able to switch devices on and off automatically, an LCD that provides clear and real time feedback, and most importantly a system that remains stable during continuous operation. The design also makes it easy to add new sensors and features.

Conclusion

The project in its current state does not meet all the goals which were initially established, but it establishes a foundation where the desired features can be implemented given more time and effort in the future. With additional time, the project can be expanded to include Wi-fi connectivity, smart phone app integration, cloud-based data logging, and additional sensors.

The current prototype proves that smart home automation can be achieved using accessible hardware and beginner-friendly technology. Beyond meeting its initial goals, the project provides a scalable framework for future students and researchers to be able and build upon.

References

— *Smart home Kit documentation.* (2026). Keystudio.com.
<https://docs.keystudio.com/projects/KS0085/en/latest/>